1What is C#? Difference between C# and .NET?

C# is a modern, object-oriented programming language developed by Microsoft that runs on the .NET platform. It is used to build applications such as web apps, desktop apps, APIs, and enterprise systems.

.NET, on the other hand, is a development framework or platform that provides the runtime (CLR), libraries, and tools required to build and run applications. C# is a language, while .NET is the environment that executes C# code.

🧠 Easy to Remember

C# = Language
.NET = Platform

👉 You write code in C#
👉 .NET runs it

🎯 Analogy

C# is like **English language**
.NET is like the **school system** where English is taught and used.

Language vs Environment.

---

2 What is OOPS? What are the main concepts?

✅ Interview-Ready Answer

OOPS (Object-Oriented Programming System) is a programming paradigm that organizes software design around objects rather than functions and logic. It models real-world entities using classes and objects to improve modularity, reusability, and maintainability.

The four main principles of OOPS are:

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

🧠 Easy to Remember

Remember: **E A I P**

Encapsulation
Abstraction
Inheritance
Polymorphism

🎯 Analogy

Think of OOPS like designing a car system:

Car is an object
Engine, brakes are parts
Different car models share common features
Same accelerator works differently in electric vs petrol car

---

3 Advantages of OOPS

✅ Interview-Ready Answer

OOPS improves code reusability, maintainability, scalability, and security. It allows modular development using classes and objects. Inheritance reduces code duplication, encapsulation protects data, and polymorphism increases flexibility.

🧠 Easy to Remember

OOPS gives:

- Reusability
- Security
- Flexibility
- Easy Maintenance

🎯 Analogy

Like building with LEGO blocks.

You reuse pieces, combine them differently, and fix one block without breaking the whole structure.

---

4 Limitations of OOPS

✅ Interview-Ready Answer

OOPS can increase program complexity due to excessive abstraction and inheritance. It may consume more memory compared to procedural programming. For small applications, OOPS may feel unnecessary and over-engineered.

🧠 Easy to Remember

More structure → More complexity
Good for large systems, not ideal for very small ones.

🎯 Analogy

Using a full project management system to plan a birthday party.
Too heavy for small tasks.

5️⃣ What are Classes and Objects?

✅ Interview-Ready Answer

A class is a blueprint or template used to create objects. It defines properties (data) and methods (behavior). An object is an instance of a class that represents a real-world entity and contains actual values.

🧠 Easy to Remember

Class = Blueprint
Object = Real Thing created from blueprint

🎯 Analogy

Class = House design plan
Object = Actual house built from that design

One design → many houses

If interviewer goes deeper, next questions will be:

- What is encapsulation with example?
- What is abstraction in C#?
- Difference between abstraction and encapsulation?
- What is virtual method?
- What is method overloading vs overriding?

Ask next 🔥

Q. What are the types of classes in C#? Q. Is it possible to prevent object creation of a class in C#? Q. What is Property? Q. What is the difference between Property and Function? Q. What are Namespaces?

Good Ashok 👏
Let's answer in **interview-ready + easy memory + analogy** format.

---

1 What are the types of classes in C#?

✅ Interview-Ready Answer

In C#, there are several types of classes:

- **Abstract class** – Cannot be instantiated and may contain abstract methods.
- **Sealed class** – Cannot be inherited.
- **Static class** – Cannot be instantiated and contains only static members.
- **Partial class** – Class definition can be split across multiple files.
- **Nested class** – A class defined inside another class.

🧠 Easy to Remember

Remember:
A S S P N

Abstract
Sealed
Static
Partial
Nested

🎯 Analogy

Think of class types like building rules:

- Abstract → Blueprint only, cannot build directly
- Sealed → Final building, no extension allowed
- Static → Utility building, no personal copy
- Partial → Blueprint in multiple pages
- Nested → Small room inside big house

---

2 Is it possible to prevent object creation of a class in C#?

✅ Interview-Ready Answer

Yes, we can prevent object creation in C# by:

- Declaring the class as **abstract**
- Declaring the class as **static**
- Making the constructor **private**

A static class cannot be instantiated at all, and a private constructor prevents external object creation.

🧠 Easy to Remember

No object creation by:

- Abstract
- Static
- Private constructor

🎯 Analogy

Private constructor is like locking the main door — nobody can enter and create an instance.

---

3 What is a Property?

✅ Interview-Ready Answer

A property in C# is a member that provides controlled access to a private field using get and set accessors. It allows encapsulation by restricting how data is accessed or modified.

🧠 Easy to Remember

Property = Controlled variable access

It protects internal data.

🎯 Analogy

Property is like a bank locker.

You cannot directly touch the money (field).
You must go through the bank system (get/set).

---

4 Difference between Property and Function

✅ Interview-Ready Answer

A property is used to get or set the value of a field, while a function (method) is used to perform actions or operations.

Properties generally represent state (data), whereas functions represent behavior (logic).

🧠 Easy to Remember

Property = Data
Function = Action

🎯 Analogy

Property = Speed of car
Function = Accelerate()

Speed stores value.
Accelerate performs action.

---

5 What are Namespaces?

✅ Interview-Ready Answer

A namespace in C# is a container used to organize related classes, interfaces, and other types. It helps avoid name conflicts and improves code structure in large applications.

🧠 Easy to Remember

Namespace = Folder for classes

🎯 Analogy

Namespace is like a folder in your computer.

Two files can have the same name if they are in different folders.

Next interview level question will be:

- What is constructor?
- What is difference between interface and abstract class?
- What is static keyword?
- What is access modifier?

Ask next 🔥

What is Inheritance? When to use Inheritance? Q. What are the different types of Inheritance? Q. Does C# support Multiple Inheritance? How can you implement multiple inheritance in C#? Q. How to prevent a class from being Inherited? Q. Are private class members inherited to the derived class? Q. What is Abstraction? How to implement abstraction in real applications? Q. What is Encapsulation? How to implement encapsulation in real applications? Q. What is the difference between Abstraction and Encapsulation? Q. What is Polymorphism and what are its

types? When to use polymorphism? Q. What is Method Overloading? In how many ways a method can be overloaded?

Excellent Ashok 🔥
These are core OOPS interview questions. I'll keep answers:

- ✅ Interview-ready
- 🧠 Easy to remember
- 🎯 Simple analogy

1️⃣ What is Inheritance? When to use it?

✅ Interview Answer

Inheritance is an OOPS concept where one class (derived/child class) acquires the properties and methods of another class (base/parent class). It promotes code reusability and establishes an "is-a" relationship between classes.

We use inheritance when multiple classes share common functionality and we want to avoid code duplication.

🧠 Easy to Remember

Inheritance = Reuse + Is-A Relationship

Dog **is a** Animal.

🎯 Analogy

Father → Child

Child inherits surname and traits.

---

2 Types of Inheritance

✅ Interview Answer

There are five types of inheritance:

1. Single
2. Multilevel
3. Hierarchical
4. Multiple
5. Hybrid

In C#, classes support Single, Multilevel, and Hierarchical inheritance directly.

🧠 Easy Trick

S M H M H

---

3 Does C# support Multiple Inheritance?

✅ Interview Answer

C# does not support multiple inheritance using classes to avoid ambiguity (Diamond Problem). However, multiple inheritance can be achieved using interfaces.

🧠 Easy to Remember

Class → No
Interface → Yes

🎯 Analogy

You can inherit property from only one father (class),
But you can learn skills from multiple teachers (interfaces).

---

4 How to prevent a class from being inherited?

✅ Interview Answer

We use the `sealed` keyword to prevent a class from being inherited.

🧠 Easy

Sealed = Final

---

5 Are private members inherited?

✅ Interview Answer

Yes, private members are inherited but they cannot be accessed directly in the derived class.
They can only be accessed through public or protected methods of the base class.

🧠 Easy

Inherited? → Yes
Accessible? → No

---

6 What is Abstraction? How to implement?

✅ Interview Answer

Abstraction means hiding internal implementation details and showing only essential features. In C#, abstraction is implemented using abstract classes and interfaces.

In real applications, we use abstraction in service layers, repository patterns, and APIs to expose only required functionality.

🧠 Easy

Hide complexity, show essentials.

🎯 Analogy

ATM machine.
You withdraw money without knowing banking logic.

7️⃣ What is Encapsulation? How to implement?

✅ Interview Answer

Encapsulation means wrapping data and methods into a single unit and restricting direct access to internal data. It is implemented using access modifiers (private, protected, public) and properties.

In real applications, we keep fields private and expose them through properties.

🧠 Easy

Data protection.

🎯 Analogy

Capsule medicine — ingredients hidden inside.

---

8 Difference between Abstraction and Encapsulation

✅ Interview Answer

Abstraction focuses on hiding implementation details and exposing functionality.
Encapsulation focuses on restricting access to data.

Abstraction is about "what to show".
Encapsulation is about "how to protect".

🧠 Easy Table

Abstraction → Hide implementation
Encapsulation → Hide data

---

9 What is Polymorphism? Types? When to use?

✅ Interview Answer

Polymorphism means "many forms". It allows methods to behave differently based on context.

Types:

1. Compile-time Polymorphism (Method Overloading)
2. Runtime Polymorphism (Method Overriding)

We use polymorphism when different classes need to implement the same method differently.

🧠 Easy

Overloading → Same method, different parameters
Overriding → Same method, different behavior

🎯 Analogy

Same button, different results:

Call button on phone
Call method in programming

🔟 What is Method Overloading? How many ways?

✅ Interview Answer

Method Overloading is compile-time polymorphism where multiple methods have the same name but different parameters.

A method can be overloaded by:

1. Changing number of parameters
2. Changing type of parameters
3. Changing order of parameters

Return type alone cannot overload a method.

🧠 Easy

Name same
Parameters different

🔥 Ashok, next level interviewer may now ask:

- What is Diamond Problem?
- What is virtual keyword?
- What is override vs new?
- What is difference between interface and abstract class?
- What is constructor chaining?

Ask next.

. When should you use method overloading in real applications? Q. If two methods are same except return type, then methods are overloaded or what willl happen? Q. What is the difference between Overloading and Overriding? Q. What is the use of Overriding? When should I override the method in real applications? Q. If a method is marked as virtual, do we must have to "override" it from the child class? Q. What is the difference between Method Overriding and Method Hiding? Q. What is the difference between an Abstract class and an Interface (atleast 4)? Q. When to use Interface and when Abstract class in real applications?

Excellent Ashok 🔥
Now we are entering **real interview depth level**.
I'll keep answers sharp, professional, and easy to revise.

1️⃣ When should you use Method Overloading in real applications?

✅ Interview Answer

Method overloading should be used when the same operation needs to be performed with different types or numbers of inputs. It improves readability and makes the API intuitive.

For example, in real applications, we overload methods like:

- `Add(int a, int b)`
- `Add(double a, double b)`
- `Add(int a, int b, int c)`

It is commonly used in constructors, logging methods, and service methods where input variations exist.

🧠 Easy to Remember

Same action
Different input

🎯 Analogy

Think of a payment system:

Pay by cash
Pay by card
Pay by UPI

Same action → Pay
Different inputs → Different parameters

---

2 If two methods differ only by return type, are they overloaded?

✅ Interview Answer

No. Methods cannot be overloaded based only on return type. The compiler identifies overloaded methods using the method signature, which includes the method name and parameters — not the return type.

If only the return type differs, it results in a compile-time error.

🧠 Easy

Return type is NOT part of method signature.

---

3 Difference between Overloading and Overriding

✅ Interview Answer

Overloading	Overriding
Compile-time polymorphism	Runtime polymorphism
Same method name, different parameters	Same method name and parameters
No inheritance required	Requires inheritance
Resolved at compile time	Resolved at runtime

🧠 Easy

Overloading → Same name, different input
Overriding → Same name, different behavior

---

4 What is the use of Overriding? When to use in real applications?

✅ Interview Answer

Method overriding allows a derived class to provide a specific implementation of a method already defined in the base class. It enables runtime polymorphism.

We override methods when different child classes need different behavior for the same method.

Example:
A base class `Payment` has a method `ProcessPayment()`.
Derived classes like `CreditCardPayment` and `UPIPayment` override it with their own logic.

🧠 Easy

Base defines rule
Child modifies behavior

---

5 If a method is virtual, must we override it?

✅ Interview Answer

No, overriding a virtual method is optional. If the derived class does not override it, the base class implementation will be used.

🧠 Easy

Virtual = Allow override
Not mandatory

---

6 Difference between Method Overriding and Method Hiding

✅ Interview Answer

Method Overriding uses `virtual` and `override` keywords and supports runtime polymorphism.

Method Hiding uses the `new` keyword and hides the base class method instead of overriding it.

Overriding → Runtime decision
Hiding → Compile-time decision

🧠 Easy

Override = Replace
New = Hide

---

7 Difference between Abstract Class and Interface (at least 4)

✅ Interview Answer

Abstract Class	Interface
Can have both abstract and non-abstract methods	Only method declarations (before C# 8)
Can have constructors	Cannot have constructors
Supports access modifiers	Members are public by default
Supports fields	Cannot have instance fields
Single inheritance only	Multiple interfaces allowed

🧠 Easy

Abstract = Partial implementation
Interface = Full contract

---

8 When to use Interface and when Abstract class?

✅ Interview Answer

Use Interface when:

- You need multiple inheritance
- You want to define a contract
- Different classes share behavior but not hierarchy

Use Abstract Class when:

- Classes share common base functionality
- You want to provide default implementation
- There is a strong "is-a" relationship

🎯 Real Application Example

In your CureMD system:

`IRepository` → Interface (contract)

`BaseRepository` → Abstract class (common DB logic)

`PatientRepository` → Concrete implementation

🧠 Final Easy Rule

Interface → What to do
Abstract → How partially to do
Class → Complete implementation

🔥 Ashok, next interviewer will go toward:

- What is SOLID principle?
- What is Dependency Injection?
- What is sealed method?
- What is covariance & contravariance?
- What is extension method?
- What is delegate?

Tell me next topic.

Why to create Interfaces in real applications? Q. Can we define body of Interfaces methods? When to define methods in Interfaces? Q. Can you create an instance of an Abstract class or an Interface? Q. Do Interface can have a Constructor? Q. Do abstract class have Constructors in C#? Q. What is the difference between abstraction and abstract class? Q. Can Abstract class be Sealed or Static in C#? Q. Can you declare abstract methods as private in C#? Q. Does Abstract class support multiple Inheritance? Q. What are Access Specifiers?

Very good Ashok 👏🔥
These are strong OOPS depth questions. I'll keep answers:

- ✅ Interview ready

- 🧠 Easy to remember
- 🎯 Simple understanding

1️⃣ Why create Interfaces in real applications?

✅ Interview Answer

Interfaces are created to define a contract that multiple classes must follow. They promote loose coupling, flexibility, and testability.

In real applications, interfaces are heavily used in Dependency Injection, Repository patterns, service layers, and unit testing.

For example, `IPaymentService` can have multiple implementations like `CreditCardPaymentService` or `UPIPaymentService`.

🧠 Easy

Interface = Contract

Loose coupling + Flexibility.

2️⃣ Can we define body in Interface methods?

✅ Interview Answer

Before C# 8, interface methods could only have declarations.
From C# 8 onwards, interfaces can have default method implementations.

We define methods in interfaces when we want to enforce a contract across multiple classes.

🧠 Easy

Old C# → No body
C# 8+ → Default implementation allowed

---

3 Can you create instance of Abstract class or Interface?

✅ Interview Answer

No, we cannot create an instance of an abstract class or an interface directly because they are incomplete types.

However, we can reference them using a derived class object.

Example:

```
IPayment payment = new CreditCardPayment();
```

🧠 Easy

Direct object? ❌
Reference type? ✅

---

4 Do Interfaces have Constructors?

✅ Interview Answer

No, interfaces cannot have constructors because they cannot be instantiated.

🧠 Easy

No object → No constructor

---

5 Do Abstract classes have Constructors?

✅ Interview Answer

Yes, abstract classes can have constructors. These constructors are called when a derived class object is created.

They are mainly used to initialize common fields.

🧠 Easy

Abstract class → Yes constructor
Interface → No constructor

---

6 Difference between Abstraction and Abstract Class

✅ Interview Answer

Abstraction is a concept of hiding implementation details and showing only essential features.

Abstract class is a mechanism in C# used to implement abstraction.

Abstraction = Concept
Abstract class = Implementation tool

🧠 Easy

Abstraction → Idea
Abstract class → Way to achieve it

---

7 Can Abstract class be Sealed or Static?

✅ Interview Answer

No.

- Abstract class cannot be sealed because sealed prevents inheritance.
- Abstract class cannot be static because static classes cannot be inherited.

Both contradict abstraction.

🧠 Easy

Abstract → Must inherit
Sealed → Cannot inherit
Static → Cannot inherit

Conflict ❌

---

8 Can abstract methods be private?

✅ Interview Answer

No. Abstract methods cannot be private because they must be overridden in derived classes.
Private methods are not accessible outside the class.

🧠 Easy

Override required → Must be visible
Private → Not visible
So ❌

---

9 Does Abstract class support multiple inheritance?

✅ Interview Answer

No. C# does not support multiple inheritance with classes, including abstract classes.

However, a class can implement multiple interfaces.

🧠 Easy

Class → Single inheritance
Interface → Multiple

---

10 What are Access Specifiers?

✅ Interview Answer

Access specifiers define the accessibility level of classes and members.

Main access specifiers in C#:

- Public
- Private
- Protected
- Internal
- Protected Internal
- Private Protected

They help implement encapsulation and security.

🧠 Easy

Access specifiers = Security levels

🔥 Ashok, next level interviewer may move toward:

- What is virtual vs abstract?
- What is sealed method?
- What is static constructor?
- What is readonly vs const?
- What is Dependency Injection?
- What is SOLID?

Tell me next topic 🚀

What is internal access modifier? Show example. Q. What is the default access modifier in a class? Q. What is Boxing and Unboxing? Where to use Boxing and Unboxing in real applications? Q. Which one is explicit Boxing or Unboxing? Q. Is Boxing and Unboxing good for performance? Q. What are the basic string operations in C#? Q. What is the difference between “String” and “StringBuilder”? Q. When to use String and when StringBuilder in real applications? Q. What is String Interpolation in C#? Q. What are the Loop types in C#? When to use what in real applications?

Very good Ashok 👏🔥
These are solid .NET fundamentals. Let’s go one by one — interview-ready and easy to revise.

1️⃣ What is internal access modifier? Show example.

✅ Interview Answer

The `internal` access modifier restricts access to members within the same assembly (project). It cannot be accessed from another project unless explicitly allowed using `InternalsVisibleTo`.

Example:

```
internal class EmployeeService
{
    internal void ProcessSalary()
    {
        Console.WriteLine("Salary Processed");
    }
}
```

This class and method can only be accessed inside the same project.

🧠 Easy

Internal = Same Project Access Only

---

2 What is the default access modifier in a class?

✅ Interview Answer

- For top-level classes → Default is `internal`.
- For class members → Default is `private`.

🧠 Easy

Class → Internal
Members → Private

---

3 What is Boxing and Unboxing?

✅ Interview Answer

Boxing is the process of converting a value type (like int) into an object type.
Unboxing is the process of converting an object back into a value type.

Example:

```
int num = 10;
object obj = num;      // Boxing
int n = (int)obj;      // Unboxing
```

🧠 Easy

Value → Object = Boxing
Object → Value = Unboxing

---

4 Which one is explicit — Boxing or Unboxing?

✅ Interview Answer

Boxing is implicit.
Unboxing is explicit because it requires casting.

🧠 Easy

Boxing → Automatic
Unboxing → Cast required

---

5 Is Boxing/Unboxing good for performance?

✅ Interview Answer

No. Boxing and unboxing impact performance because:

- Boxing allocates memory on heap.
- Unboxing requires type casting.
- It increases GC pressure.

In high-performance applications, it should be avoided.

🧠 Easy

More heap usage → Slower performance

---

6 Basic String Operations in C#

✅ Interview Answer

Common string operations:

- Concatenation (+)
- Length
- ToUpper() / ToLower()
- Substring()
- Replace()
- Split()
- Trim()
- Contains()
- IndexOf()

🧠 Easy

Strings = Search + Modify + Compare

---

7 Difference between String and StringBuilder

✅ Interview Answer

String	StringBuilder
Immutable	Mutable
Creates new object on modification	Modifies same object
Slower for frequent changes	Faster for multiple modifications

---

🧠 Easy

String → Fixed
StringBuilder → Editable

---

8 When to use String vs StringBuilder?

✅ Interview Answer

Use String:

- When few modifications are needed
- For small text operations

Use StringBuilder:

- When performing many string manipulations inside loops
- Large dynamic string building (like logs, reports)

🎯 Real Example

Building 1000-line report inside loop → Use StringBuilder

---

9 What is String Interpolation?

✅ Interview Answer

String interpolation allows embedding variables directly inside a string using `$` and `{}`.

Example:

```
string name = "Ashok";
int age = 30;
string message = $"My name is {name} and I am {age} years old.";
```

It improves readability.

🧠 Easy

$"Hello {name}"

Cleaner than concatenation.

🔟 Loop Types in C#

✅ Interview Answer

C# supports:

1. for
2. foreach
3. while
4. do-while

When to Use?

- `for` → When number of iterations is known
- `foreach` → When iterating collections
- `while` → When condition-based looping
- `do-while` → When loop must execute at least once

🧠 Easy

for → Count based
foreach → Collection based
while → Condition based
do-while → At least once

🔥 Ashok, next interviewer may go to:

- What is var vs dynamic?
- What is ref vs out?
- What is const vs readonly?
- What is exception handling?
- What is IEnumerable vs IQueryable?
- What is async/await?

Tell me next topic 🚀

What is the difference between “continue” and “break” statement? Q. What are the alternative ways of writing if-else conditions? When to use what? Q. How to implement Exception Handling in C#? Q. Can we execute multiple Catch blocks? Q. When to use Finally in real applications? Q. Can we have only “Try” block without “Catch” block in real applications? Q. What is the difference between Finally and Finalize? Q. What is the difference between “throw ex” and “throw”? Which one to use in real applications? Q. Explain Generics in C#? When and why to use them in real applications? Q. What are Collections in C# and what are their types?

Excellent Ashok 🔥
Now we are in **core .NET practical interview zone**.
Let’s answer clearly and professionally.

1️⃣ Difference between `continue` and `break`

✅ Interview Answer

- `break` immediately terminates the loop and exits it.
- `continue` skips the current iteration and moves to the next iteration of the loop.

🧠 Easy

Break → Stop loop
Continue → Skip current iteration

🎯 Example

If checking numbers 1–10:

- break at 5 → loop stops at 5
- continue at 5 → skips 5 but continues till 10

---

2 Alternative ways of writing if-else

✅ Interview Answer

Alternative ways:

1. Ternary operator (`?:`)
2. Switch statement
3. Switch expression (C# 8+)
4. Pattern matching

Example:

Ternary:

```
string result = age > 18 ? "Adult" : "Minor";
```

Switch Expression:

```
var result = age switch
{
    > 18 => "Adult",
    _ => "Minor"
};
```

🧠 When to use?

- Ternary → Simple condition
- Switch → Multiple fixed values
- Pattern matching → Complex condition

---

3 How to implement Exception Handling in C#?

✅ Interview Answer

Exception handling is implemented using:

- try
- catch
- finally
- throw

Example:

```
try
{
    int result = 10 / 0;
}
catch (DivideByZeroException ex)
{
    Console.WriteLine(ex.Message);
}
finally
{
    Console.WriteLine("Cleanup done");
}
```

🧠 Easy

Try → Risky code
Catch → Handle error
Finally → Cleanup

---

4 Can we execute multiple catch blocks?

✅ Interview Answer

Yes. Multiple catch blocks can be used to handle different exception types. The most specific exception should be placed first.

---

5 When to use Finally?

✅ Interview Answer

Finally block is used for cleanup operations such as:

- Closing database connections
- Releasing file streams
- Disposing unmanaged resources

It executes whether exception occurs or not.

🎯 Real Example

Closing SQL connection in CureMD API.

---

6 Can we have only Try without Catch?

✅ Interview Answer

No. A try block must have at least one catch or finally block. Try alone is invalid.

---

7 Difference between Finally and Finalize

✅ Interview Answer

Finally	Finalize
Used in exception handling	Used in garbage collection
Executes immediately after try/catch	Called by GC before object destruction
Used for cleanup logic	Used to release unmanaged resources

🧠 Easy

Finally → Manual cleanup
Finalize → Garbage collector cleanup

---

8 Difference between `throw ex` and `throw`

✅ Interview Answer

- `throw` preserves original stack trace.
- `throw ex` resets stack trace.

In real applications, always use `throw` to maintain debugging information.

🧠 Easy

throw → Keeps original error path ✅
throw ex → Loses original path ❌

---

9 Explain Generics in C#

✅ Interview Answer

Generics allow writing type-safe, reusable code by defining classes, methods, or collections with a placeholder type.

Example:

```
List<int>
List<string>
```

Generics improve:

- Type safety
- Performance (no boxing)
- Code reusability

🎯 Real Application

Repository pattern:

```
IRepository<T>
```

🧠 Easy

Write once → Use for any type

🔟 What are Collections in C#?

✅ Interview Answer

Collections are data structures used to store and manage groups of objects.

Types:

1. Non-Generic Collections
   - ArrayList
   - Hashtable
2. Generic Collections
   - List<T>
   - Dictionary<TKey, TValue>
   - Queue<T>
   - Stack<T>
3. Concurrent Collections
   - ConcurrentDictionary

🧠 Easy

Collections = Data containers

List → Ordered
Dictionary → Key-value
Queue → FIFO
Stack → LIFO

🔥 Ashok, next interviewer will likely go toward:

- What is IEnumerable vs ICollection vs IList?
- What is LINQ?
- What is async/await?
- What is Task?
- What is Dependency Injection?
- What is SOLID principles?

Which topic next? 🚀

Q. What is the difference between Array and ArrayList (atleast 2)? Q. What is the difference between Arraylist and Hashtable? Q. What is the difference between List and Dictionary Collections? Q. What is IEnumerable in C#? Q. What is the difference between IEnumerable and IEnumerator in C#? Q. What is the difference between IEnumerable and IQueryable in C#? Why to use IQueryable in sql queries? What is a Constructor? When to use constructor in real applications? Q. What are the types of constructor? Q. What is Default constructor? Q. What is Parameterized constructor?

Excellent Ashok 🔥
Now we're entering **Collections + LINQ + Constructors core area** — very important for interviews.

I'll keep answers crisp and interview-ready.

1️⃣ Difference between Array and ArrayList (at least 2)

✅ Interview Answer

Array	ArrayList
Fixed size	Dynamic size
Type-safe (stores specific type)	Stores objects (non-generic)
Better performance	Slower (boxing/unboxing)

Array is strongly typed, whereas ArrayList stores elements as object type.

🧠 Easy

Array → Fixed + Type-safe
ArrayList → Dynamic + Not type-safe

---

2 Difference between ArrayList and Hashtable

✅ Interview Answer

ArrayList	Hashtable
Stores values only	Stores key-value pairs
Access by index	Access by key
Ordered collection	No guaranteed order

🧠 Easy

ArrayList → List
Hashtable → Dictionary

---

3 Difference between List and Dictionary

✅ Interview Answer

List<T>	Dictionary<TKey, TValue>
Stores items sequentially	Stores key-value pairs
Access using index	Access using key
Allows duplicates	Keys must be unique

🧠 Easy

List → Order matters
Dictionary → Key matters

---

4 What is IEnumerable?

✅ Interview Answer

`IEnumerable` is an interface that allows iteration over a collection using `foreach`. It defines a single method `GetEnumerator()`.

It supports forward-only, read-only iteration.

🧠 Easy

IEnumerable → Can iterate

---

5 Difference between IEnumerable and IEnumerator

✅ Interview Answer

IEnumerable	IEnumerator
Provides GetEnumerator()	Provides MoveNext(), Current
Represents collection	Represents cursor
Used in foreach	Controls iteration

🧠 Easy

IEnumerable → Collection
IEnumerator → Pointer

---

6 Difference between IEnumerable and IQueryable

✅ Interview Answer

IEnumerable	IQueryable
Works with in-memory data	Works with remote data (like DB)
Executes query immediately	Executes query in database
Filters data after fetching	Filters data before fetching

Why use IQueryable in SQL?

Because IQueryable converts LINQ queries into SQL queries and executes them in the database, improving performance by reducing unnecessary data retrieval.

🧠 Easy

IEnumerable → Filter in memory
IQueryable → Filter in database

Very important for EF Core interviews 🔥

---

7 What is a Constructor?

✅ Interview Answer

A constructor is a special method of a class that is automatically called when an object is created. It is used to initialize class fields or allocate resources.

🎯 Real Application

Injecting dependencies in ASP.NET Core controllers.

🧠 Easy

Constructor = Object initializer

---

8 Types of Constructors

✅ Interview Answer

1. Default constructor
2. Parameterized constructor
3. Copy constructor
4. Static constructor
5. Private constructor

---

9 What is Default Constructor?

✅ Interview Answer

A default constructor is a constructor without parameters. If no constructor is defined, C# automatically provides one.

Example:

```
public class Employee
{
    public Employee()
    {
        Console.WriteLine("Object Created");
    }
}
```

🧠 Easy

No parameters → Default constructor

🔟 What is Parameterized Constructor?

✅ Interview Answer

A parameterized constructor accepts parameters and is used to initialize fields with specific values at object creation.

Example:

```
public class Employee
{
    public string Name;

    public Employee(string name)
    {
        Name = name;
    }
}
```

🧠 Easy

Constructor with input → Parameterized

🔥 Ashok, next interviewer will likely go to:

- Static constructor?
- Destructor?
- Dependency Injection?
- LINQ?
- async/await?
- Task vs Thread?
- IEnumerable vs IList vs ICollection?
- What is deferred execution?

Tell me next topic 🚀

What is Static constructor? What is the use in real applications? Q. Can we have parameters or access modifier in static constructor? Q. What is Copy constructor? Q. What is Private

constructor? What is the use? Q. What is Constructor overloading? Q. What is Destructor? Q. Can you create object of class with private constructor in C#? Q. If base class and child class both have constructor which one will be called first, when derived class object is created? Q. What is a Method in C#? Q. What is the difference between Pass by Value and Pass by Reference Parameters? Q. How to return more than one value from a method in C#? Q. What is the difference between “out” and “ref” parameters?

Excellent Ashok 🔥
Now we’re covering **constructors + memory + parameters** — very important for senior interviews.

I’ll keep everything structured and easy to remember.

1️⃣ What is Static Constructor? Use in real applications?

✅ Interview Answer

A static constructor is a special constructor used to initialize static members of a class. It is automatically called only once, before the first instance is created or any static member is accessed.

It is mainly used for:

- Initializing static variables
- Loading configuration
- Initializing static resources

Example:

```
public class Logger
{
    static Logger()
    {
        Console.WriteLine("Static constructor called");
    }
}
```

🧠 Easy

Static constructor → Runs once
Used for static initialization

---

2 Can static constructor have parameters or access modifier?

✅ Interview Answer

No.

- Static constructors cannot have parameters.
- Static constructors cannot have access modifiers.
- They are always private by default.

🧠 Easy

No parameters ❌
No public/private ❌

---

3 What is Copy Constructor?

✅ Interview Answer

A copy constructor creates a new object by copying values from another object of the same class.

Example:

```
public class Employee
{
    public string Name;

    public Employee(Employee emp)
    {
        Name = emp.Name;
    }
}
```

Used when we want to duplicate object state.

---

4 What is Private Constructor? Use?

✅ Interview Answer

A private constructor restricts object creation from outside the class.

Used in:

- Singleton pattern
- Utility classes
- Static classes

🧠 Easy

Private constructor → No external object creation

---

5 What is Constructor Overloading?

✅ Interview Answer

Constructor overloading means defining multiple constructors in the same class with different parameters.

It provides flexibility while creating objects.

🧠 Easy

Multiple constructors → Different inputs

---

6 What is Destructor?

✅ Interview Answer

A destructor is a special method used to release unmanaged resources before object destruction. It is called automatically by the garbage collector.

Example:

```
~Employee()
{
    Console.WriteLine("Object destroyed");
}
```

🧠 Easy

Destructor → Cleanup before GC

---

7 Can you create object of class with private constructor?

✅ Interview Answer

No, not directly from outside the class.

However, object creation is possible:

- Inside the same class
- Through static methods (Factory pattern)

---

8 Constructor Call Order (Base & Child)

✅ Interview Answer

When a derived class object is created:

1. Base class constructor is called first
2. Then derived class constructor

Because base class must initialize first.

🧠 Easy

Parent first → Child next

---

9 What is a Method in C#?

✅ Interview Answer

A method is a block of code that performs a specific task. It contains a return type, method name, parameters, and body.

Example:

```
public int Add(int a, int b)
{
    return a + b;
}
```

---

10 Difference between Pass by Value and Pass by Reference

✅ Interview Answer

Pass by Value	Pass by Reference
Copy of variable passed	Actual variable reference passed
Changes not reflected	Changes reflected outside
Default behavior	Uses ref/out

🧠 Easy

Value → Copy
Reference → Original

1️⃣1️⃣ How to return more than one value?

✅ Interview Answer

Ways to return multiple values:

1. Using `out` parameters
2. Using Tuple
3. Using custom class
4. Using struct

Example (Tuple):

```
public (int, int) GetNumbers()
{
    return (10, 20);
}
```

1️⃣2️⃣ Difference between `out` and `ref`

✅ Interview Answer

ref	out
Must initialize before passing	No need to initialize
Used for read & write	Used mainly for output
Less strict	Must assign inside method

🧠 Easy

ref → Already has value
out → Must assign inside method

🔥 Ashok, next interview level:

- What is async/await?
- What is Task?
- What is Thread vs Task?
- What is sealed method?
- What is readonly vs const?
- What is dependency injection?
- What is IDisposable?

Tell me next 🚀

What is “params” keyword? When to use params keyword in real applications? Q. What are optional parameters in a method? Q. What are named parameters in a method? Q. What are Extension Methods in C#? When to use extension methods in real applications? Q. What are Delegates in C#? When to use delegates in real applications? Q. What are Multicast Delegates? Q. What are Anonymous Delegates in C#? Q. What are the differences between Events and Delegates? Q. What is ‘this’ keyword in C#? When to use it in real applications? Q. What is the purpose of “using” keyword in C#? Q. Can we use Using keyword with other classes apart from DBConnection? Q. What is the difference between “is” and “as” operators? Q. What is the difference between “Readonly” and “Constant” variables (atleast 3)? Q. What is “Static" class? When to use static class in real application? Q. What is the difference between “var” and “dynamic” in C#? Q. What is Enum keyword used for? Q. Is it possible to inherit Enum in C#? Q. What is the use of Yield keyword in C#?

Excellent Ashok 🔥
This is now **advanced C# interview territory** (delegates, events, memory, runtime behavior).
I’ll keep answers crisp + interview ready.

1️⃣ What is `params` keyword?

✅ Interview Answer

The `params` keyword allows a method to accept a variable number of arguments as an array.

Example:

```
public int Add(params int[] numbers)
{
    return numbers.Sum();
}
```

Used when number of inputs is unknown.

🎯 Real Use

Logging methods:

```
Log(string message, params object[] args)
```

🧠 Easy

params → Variable number of arguments

---

2 What are Optional Parameters?

✅ Interview Answer

Optional parameters allow assigning default values to method parameters.

```
public void Print(string name = "Guest")
```

If no argument is passed, default value is used.

---

3 What are Named Parameters?

✅ Interview Answer

Named parameters allow passing arguments using parameter names.

```
Print(name: "Ashok");
```

Improves readability and flexibility in argument order.

---

4 What are Extension Methods?

✅ Interview Answer

Extension methods allow adding new methods to an existing class without modifying it. They are defined as static methods inside static classes using `this` keyword.

Example:

```
public static class StringExtensions
{
    public static bool IsEmail(this string str)
    {
        return str.Contains("@");
    }
}
```

🎯 Real Use

Adding helper methods to models or DTOs.

🧠 Easy

Add method without modifying original class.

5️⃣ What are Delegates?

✅ Interview Answer

A delegate is a type-safe function pointer that holds reference to a method.

Used for:

- Callback methods
- Event handling
- Passing methods as parameters

Example:

```
public delegate void PrintDelegate(string msg);
```

---

6 What are Multicast Delegates?

✅ Interview Answer

Multicast delegates can hold multiple method references and invoke them in sequence.

Example:

```
del += Method1;
del += Method2;
```

Used in event handling.

---

7 What are Anonymous Delegates?

✅ Interview Answer

Anonymous delegates are delegates without a named method.

Example:

```
delegate(string msg)
{
    Console.WriteLine(msg);
};
```

Now mostly replaced by Lambda expressions.

---

8 Difference between Events and Delegates

✅ Interview Answer

Delegate	Event
Can be invoked directly	Can be invoked only inside declaring class
Represents method reference	Built on delegate
No access restriction	Provides encapsulation

Event is a restricted delegate.

---

9 What is `this` keyword?

✅ Interview Answer

`this` refers to the current instance of a class.

Used for:

- Differentiating fields and parameters
- Calling another constructor
- Extension methods

---

10 Purpose of `using` keyword?

✅ Interview Answer

The `using` statement ensures proper disposal of unmanaged resources by calling `Dispose()` automatically.

Example:

```
using (SqlConnection con = new SqlConnection())
{
}
```

---

11 Can we use `using` with other classes?

✅ Yes.

Any class implementing `IDisposable` can be used with `using`.

Example:

- FileStream

- StreamReader
- HttpClient

---

12 Difference between `is` and `as`

✅ Interview Answer

is	as
Checks type	Casts safely
Returns true/false	Returns null if fail
No casting	Performs casting

---

13 Difference between `readonly` and `const`

✅ Interview Answer

const	readonly
Compile-time constant	Runtime constant
Must initialize at declaration	Can initialize in constructor
Implicitly static	Can be instance-level

---

14 What is Static Class?

✅ Interview Answer

A static class:

- Cannot be instantiated
- Contains only static members
- Cannot be inherited

Used for utility/helper classes.

Example: `Math`, `Convert`

---

15 Difference between `var` and `dynamic`

✅ Interview Answer

var	dynamic
Type checked at compile-time	Type checked at runtime
Strongly typed	Weakly typed
Cannot change type	Can change type

---

16 What is Enum?

✅ Interview Answer

Enum defines a set of named constants.

Example:

```
enum Status { Active, Inactive }
```

Used for fixed values like roles, status, days.

---

17 Can we inherit Enum?

❌ No.

Enums cannot be inherited because they are sealed value types.

---

18 What is `yield` keyword?

✅ Interview Answer

`yield` is used to return elements one at a time from a method without creating a full collection in memory.

Example:

```
public IEnumerable<int> GetNumbers()
{
    yield return 1;
    yield return 2;
}
```

Used for lazy loading and large data processing.

🧠 Easy

yield → Return one-by-one (lazy execution)

🔥 Ashok, now we are ready for:

- LINQ deep questions
- async/await
- Task vs Thread
- Dependency Injection
- SOLID Principles
- Memory management & GC
- .NET Core pipeline
- Entity Framework questions

Tell me next level 🚀

What is LINQ? When to use LINQ in real applications? Q. What are the advantages & disadvantages of LINQ? Q. What is Lambda Expressions? What is the use in real applications? Q. What is the difference between First and FirstOrDefault methods in LINQ? Q. What are the important components of .NET framework? Q. What is an Assembly? What are the different types of assembly in .NET? Q. What is GAC? Q. What is Reflection? Q. What are Serialization and Deserialization? What are the types of serialization?

Excellent Ashok 🔥
Now we are in **LINQ + .NET Architecture + Runtime Concepts** — very common in senior interviews.

I'll keep everything clear and structured.

---

1 What is LINQ? When to use it?

✅ Interview Answer

LINQ (Language Integrated Query) is a feature in C# that allows querying data from different sources such as collections, databases, XML, etc., using a consistent syntax.

It provides readable and type-safe query capabilities directly inside C#.

We use LINQ in real applications for:

- Filtering data
- Sorting data
- Grouping
- Joining tables
- Database querying with Entity Framework

🎯 Real Example

```
var activePatients = patients.Where(p => p.IsActive).ToList();
```

🧠 Easy

LINQ = SQL inside C#

---

2 Advantages & Disadvantages of LINQ

✅ Advantages

- Improves readability
- Type-safe
- Reduces code
- Works with multiple data sources
- Deferred execution support

❌ Disadvantages

- Performance overhead in some cases
- Complex queries can be harder to debug
- Not always best for very heavy performance operations

🧠 Easy

Less code + More readable
But sometimes slower

---

3 What is Lambda Expression?

✅ Interview Answer

A Lambda Expression is a concise way to write anonymous functions using => operator.

Example:

```
p => p.IsActive
```

Used heavily in LINQ and delegates.

🎯 Real Use

```
var result = list.Where(x => x.Age > 18);
```

🧠 Easy

Lambda = Short anonymous method

---

4 Difference between First() and FirstOrDefault()

✅ Interview Answer

First()	FirstOrDefault()
Throws exception if no match	Returns default value
Used when data must exist	Used when data may not exist

🧠 Easy

First → Must exist
FirstOrDefault → Safe

---

5 Important Components of .NET Framework

✅ Interview Answer

Main components:

1. CLR (Common Language Runtime)
2. CTS (Common Type System)
3. CLS (Common Language Specification)
4. BCL (Base Class Library)
5. Assemblies

🧠 Easy

CLR + Libraries = .NET

---

6 What is an Assembly?

✅ Interview Answer

An assembly is a compiled output (.dll or .exe) containing:

- IL code
- Metadata
- Manifest

It is the smallest unit of deployment in .NET.

Types of Assembly

1. Private Assembly
2. Shared Assembly
3. Satellite Assembly

---

7 What is GAC?

✅ Interview Answer

GAC (Global Assembly Cache) stores shared assemblies that are used by multiple applications.

Assemblies in GAC must have a strong name.

🧠 Easy

GAC = Shared DLL storage

---

8 What is Reflection?

✅ Interview Answer

Reflection allows inspecting metadata of assemblies, types, methods, and properties at runtime.

Used in:

- Dependency Injection
- ORM frameworks
- Unit testing frameworks

Example:

```
Type t = typeof(Employee);
```

🧠 Easy

Reflection = Inspect program at runtime

---

9 What is Serialization & Deserialization?

✅ Interview Answer

Serialization is converting an object into a format (JSON, XML, Binary) for storage or transmission.

Deserialization is converting that format back into an object.

🎯 Real Example

Sending API response in JSON.

Types of Serialization

1. Binary Serialization
2. XML Serialization

3. JSON Serialization

🧠 Easy

Object → JSON = Serialization
JSON → Object = Deserialization

🔥 Ashok, next interviewer direction will likely be:

- What is async/await?
- What is Task?
- What is Thread vs Task?
- What is Dependency Injection?
- What is SOLID principles?
- What is Middleware in ASP.NET Core?
- What is Garbage Collection?

Tell me next topic 🚀

What is meant by Globalization and Localization? Q. What are Window Services? Q. What is Garbage Collection(GC)? Q. What are Generations in garbage collection? Q. What is the difference between “Dispose” and “Finalize”? Q. What is the difference between “Finalize” and “Finally” methods? Q. Can we force Garbage Collector to run? Q. What is the difference between Process and Thread? Q. Explain Multithreading? Q. What is the difference between synchronous and asynchronous programming? What is the role of Task? Q. What is the difference between Threads and Tasks? What are the advantages of Tasks over Threads? Q. What is the role of Async and Await? Q. What is the difference between DBMS and RDBMS? Q. What is a Constraint in SQL? What are the types of constraints? Q. What is the difference between Primary key and Unique key?

Excellent Ashok 🔥
Now we are covering **System Design + Memory + Multithreading + DB fundamentals** — very important for senior interviews.

I’ll keep it interview-ready + easy to revise.

1️⃣ Globalization vs Localization

✅ Interview Answer

- **Globalization** is designing an application to support multiple languages and regions.
- **Localization** is customizing the application for a specific region or language.

Example:
Global app supports multiple languages.
Localization → English → Urdu → French versions.

🧠 Easy

Globalization → Prepare for world
Localization → Adapt for one country

---

2 What are Windows Services?

✅ Interview Answer

Windows Service is a long-running background application that runs without user interaction.

Used for:

- Email sending service
- Background job processing
- Monitoring systems

It starts automatically with OS.

---

3 What is Garbage Collection (GC)?

✅ Interview Answer

Garbage Collection is an automatic memory management feature of .NET that removes unused objects from heap memory.

It prevents memory leaks.

🧠 Easy

GC = Automatic memory cleaner

---

4 Generations in GC

✅ Interview Answer

.NET GC has 3 generations:

- Gen 0 → Short-lived objects
- Gen 1 → Medium-lived objects
- Gen 2 → Long-lived objects

Most collections happen in Gen 0.

🧠 Easy

New → Gen 0
Survived → Gen 1
Long-term → Gen 2

---

5 Dispose vs Finalize

✅ Interview Answer

Dispose	Finalize
Manual cleanup	Automatic cleanup
Implements IDisposable	Called by GC
Used for unmanaged resources	Backup cleanup

Dispose is preferred.

---

6 Finalize vs Finally

✅ Interview Answer

Finalize	Finally
Called by GC	Used in exception handling
Memory cleanup	Code cleanup
Runs unpredictably	Always runs after try/catch

---

7 Can we force GC?

✅ Yes

```
GC.Collect();
```

But not recommended unless necessary.

---

8 Process vs Thread

✅ Interview Answer

Process	Thread
Independent program	Part of process
Has own memory	Shares process memory
Heavyweight	Lightweight

🧠 Easy

Process = House
Thread = Room

---

9 What is Multithreading?

✅ Interview Answer

Multithreading allows multiple threads to run concurrently within a process to improve performance and responsiveness.

Used in:

- Background tasks
- Parallel processing
- Web servers

---

10 Synchronous vs Asynchronous

✅ Interview Answer

Synchronous	Asynchronous
Executes one by one	Non-blocking
Waits for completion	Continues execution
Blocking calls	Better performance

Role of Task

Task represents asynchronous operation and runs on thread pool.

Example:

```
Task.Run(() => DoWork());
```

---

11 Threads vs Tasks

✅ Interview Answer

Thread	Task
Low-level	High-level abstraction
Manual management	Managed by thread pool
More complex	Easier & safer

Advantages of Task:

- Better performance
- Simplified async handling
- Exception handling support

---

12 Role of async & await

✅ Interview Answer

- `async` enables asynchronous method.
- `await` pauses execution until task completes without blocking thread.

Improves scalability in web apps.

---

13 DBMS vs RDBMS

✅ Interview Answer

DBMS	RDBMS
Stores data	Stores relational data
No relationships	Uses tables & relationships
Example: File system	Example: SQL Server

---

14 What is Constraint in SQL?

✅ Interview Answer

Constraint enforces rules on table data.

Types:

- Primary Key
- Foreign Key
- Unique
- Not Null
- Check
- Default

1️⃣5️⃣ Primary Key vs Unique Key

✅ Interview Answer

Primary Key	Unique Key
Only one per table	Multiple allowed
Cannot be null	Can allow one null (SQL Server)
Identifies row uniquely	Ensures uniqueness

🔥 Ashok — now you're covering almost full .NET stack fundamentals.

Next interviewer level may go into:

- What is Deadlock?
- What is Dependency Injection?
- What is SOLID?
- What is Middleware?
- What is JWT?
- What is Microservices?
- What is REST API?
- What is EF Core tracking?

Tell me next 🚀

. What are Triggers and types of triggers? Q. What is a View? Q. What is the difference between Having clause and Where clause? Q. What is Sub query or Nested query or Inner query in SQL? Q. What is Auto Increment/ Identity column in SQL Server? Q. What are Joins in SQL? Q. What are the types of Joins in SQL Server? Q. What is Self-Join? Q. What are Indexes in SQL Server? Q. What is Clustered index? Q. What is Non-Clustered index? Q. What is the difference between Clustered and Non-Clustered index?

Excellent Ashok 🔥
Now we're in **pure SQL interview mode** — very common for .NET + Backend roles.

Let's go step by step — interview ready + simple.

1️⃣ What are Triggers? Types of Triggers?

✅ Interview Answer

A Trigger is a special type of stored procedure that automatically executes when an event (INSERT, UPDATE, DELETE) occurs on a table.

Types of Triggers:

1. DML Triggers (INSERT, UPDATE, DELETE)
2. DDL Triggers (CREATE, ALTER, DROP)
3. INSTEAD OF Trigger

🎯 Real Use

- Audit logging
- Data validation
- Enforcing business rules

🧠 Easy

Trigger = Automatic action on table event

2️⃣ What is a View?

✅ Interview Answer

A View is a virtual table created from a SELECT query. It does not store data physically but shows data from one or more tables.

Used for:

- Security
- Simplifying complex queries
- Hiding sensitive columns

🧠 Easy

View = Saved query

---

3 Difference between WHERE and HAVING

✅ Interview Answer

WHERE	HAVING
Filters rows before grouping	Filters after GROUP BY
Cannot use aggregate functions	Can use aggregate functions
Works on raw data	Works on grouped data

🧠 Easy

WHERE → Before grouping
HAVING → After grouping

---

4 What is Subquery?

✅ Interview Answer

A Subquery is a query inside another query.

Example:

```
SELECT * FROM Employees
WHERE Salary > (SELECT AVG(Salary) FROM Employees);
```

Used for filtering based on another query's result.

---

5 What is Identity (Auto Increment)?

✅ Interview Answer

Identity column automatically generates unique numeric values for each new row.

Example:

```
ID INT IDENTITY(1,1)
```

(Starts at 1, increments by 1)

Used for primary keys.

---

6 What are Joins?

✅ Interview Answer

Joins are used to combine rows from two or more tables based on related columns.

---

7 Types of Joins in SQL Server

1. INNER JOIN
2. LEFT JOIN
3. RIGHT JOIN
4. FULL JOIN
5. CROSS JOIN

---

8 What is Self Join?

✅ Interview Answer

A Self Join is when a table is joined with itself.

Used for:

- Employee → Manager relationships

Example:

```
SELECT e.Name, m.Name
FROM Employees e
JOIN Employees m
ON e.ManagerId = m.Id;
```

---

9 What are Indexes?

✅ Interview Answer

Indexes improve query performance by allowing faster data retrieval.

They work like a book index.

---

10 What is Clustered Index?

✅ Interview Answer

Clustered index determines the physical order of data in a table.

- Only one per table
- Data stored in sorted order

Primary key is clustered by default.

1️⃣1️⃣ What is Non-Clustered Index?

✅ Interview Answer

Non-clustered index creates a separate structure that points to table data.

- Multiple allowed per table
- Does not change physical order

1️⃣2️⃣ Difference between Clustered & Non-Clustered

✅ Interview Answer

Clustered Index	Non-Clustered Index
Sorts actual data	Separate structure
Only one allowed	Multiple allowed
Faster for range queries	Faster for specific lookups
Large data movement on insert	Less impact on insert

🧠 Easy

Clustered → Data sorted physically
Non-Clustered → Pointer to data

🔥 Ashok — you're covering full stack fundamentals now.

Next interviewer may go into:

- What is normalization?
- What is deadlock?
- What is transaction?
- ACID properties?
- Stored procedures vs functions?

- CTE?
- Execution plan?
- Difference between DELETE, TRUNCATE, DROP?

Tell me next 🚀

How to create Clustered and Non-Clustered index in a table? " Q. In which column you will apply the indexing to optimize this query. “select id, class from student where name=”happy””?" What is the difference between Stored Procedure and Functions (at least 3)? Q. How to optimize a Stored Procedure or SQL Query? Q. What is a Cursor? Why to avoid them? Q. What is the difference between scope_identity and @@identity? Q. What is CTE in SQL Server? Q. What is the difference between Delete, Truncate and Drop commands? Q. How to get the Nth highest salary of an employee? Q. What are ACID properties? Q. What are Magic Tables in SQL Server? Q. What is MVC? Explain MVC Life cycle.

Excellent Ashok 🔥
Now we’re in **advanced SQL + ASP.NET MVC fundamentals** — very common in backend interviews.

I’ll keep answers sharp and interview-ready.

1️⃣ How to Create Clustered & Non-Clustered Index?

✅ Clustered Index

```
CREATE CLUSTERED INDEX IX_Student_Name
ON Student(Name);
```

Only one clustered index per table.

✅ Non-Clustered Index

```
CREATE NONCLUSTERED INDEX IX_Student_Class
ON Student(Class);
```

Multiple allowed.

---

2 On which column will you apply indexing?

Query:

```
SELECT id, class FROM student WHERE name = 'happy';
```

✅ Interview Answer

Index should be created on the `name` column because it is used in the WHERE condition for filtering.

Better optimization:

```
CREATE NONCLUSTERED INDEX IX_Student_Name
ON Student(Name)
INCLUDE (Id, Class);
```

This becomes a covering index.

---

3 Difference between Stored Procedure and Function (at least 3)

Stored Procedure	Function
Can return multiple result sets	Returns single value or table
Can perform INSERT/UPDATE/DELETE	Cannot modify table data
Supports TRY-CATCH	No TRY-CATCH
Can use temp tables	Limited

---

4 How to Optimize Stored Procedure / Query?

✅ Interview Answer

Ways to optimize:

- Use proper indexing

- Avoid SELECT *
- Use JOIN instead of subqueries (when possible)
- Avoid cursors
- Use SET NOCOUNT ON
- Analyze execution plan
- Avoid functions in WHERE clause
- Use proper data types

---

5 What is Cursor? Why avoid it?

✅ Interview Answer

Cursor processes rows one by one.

It is slow because:

- Row-by-row processing
- High memory usage
- Locks resources longer

Better to use set-based operations.

---

6 Difference between SCOPE_IDENTITY() and @@IDENTITY

SCOPE_IDENTITY()	@@IDENTITY
Returns last identity in same scope	Returns last identity in session
Safe from trigger side effects	Can return trigger identity
Recommended	Not recommended

---

7 What is CTE?

✅ Interview Answer

CTE (Common Table Expression) is a temporary result set defined using WITH clause.

Example:

```
WITH SalaryCTE AS
(
    SELECT * FROM Employee
)
SELECT * FROM SalaryCTE;
```

Used for recursion and improving readability.

---

8 Difference between DELETE, TRUNCATE, DROP

DELETE	TRUNCATE	DROP
Removes rows	Removes all rows	Removes table
Can use WHERE	No WHERE	Deletes structure
Logged row-by-row	Minimal logging	Removes entire object
Can rollback	Can rollback	Cannot rollback

---

9 How to get Nth Highest Salary?

✅ Using DENSE_RANK()

```
SELECT Salary
FROM (
    SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS Rank
    FROM Employee
) t
WHERE Rank = 2;
```

---

10 What are ACID properties?

✅ Interview Answer

ACID ensures database reliability:

- Atomicity → All or nothing
- Consistency → Valid state
- Isolation → No interference
- Durability → Permanent commit

1⃣1⃣ What are Magic Tables?

✅ Interview Answer

Magic tables are special tables available inside triggers:

- inserted
- deleted

Used to access affected rows.

1⃣2⃣ What is MVC?

✅ Interview Answer

MVC (Model-View-Controller) is an architectural pattern.

- Model → Business logic
- View → UI
- Controller → Handles request

MVC Lifecycle (ASP.NET MVC)

1. Request hits Routing
2. Route maps to Controller
3. Controller calls Model
4. Model processes data
5. View renders response
6. Response sent to browser

🧠 Easy

Request → Controller → Model → View → Response

🔥 Ashok — You have now covered:

✔ C# Core
✔ OOPS
✔ Advanced C#
✔ LINQ
✔ Multithreading
✔ SQL
✔ MVC

Next level topics interviewer may ask:

- What is Dependency Injection?
- What is Middleware in ASP.NET Core?
- What is JWT?
- What is REST API?
- What is SOLID?
- What is Deadlock?
- What is Microservices?
- What is EF Core tracking?

Tell me next 🚀

What are the advantages of MVC over Web Forms (atleast 3)? Q. What are the different return types of a controller Action method? Q. What are Filters and their types in MVC? Q. What is Authentication and Authorization in ASP.NET MVC? Q. What are the types of Authentication in ASP.NET MVC? Q. What is Output Caching in MVC? How to implement it? Q. What is Routing in MVC? Q. Explain Attribute Based Routing in MVC? Q. What is the difference between ViewData, ViewBag & TempData? Q. How can we pass the data from controller to view in MVC? Q. What is Partial View?

Excellent Ashok 🔥
Now we are in **ASP.NET MVC deep interview zone** — very important for backend roles.

Let’s go question by question — interview ready + simple to revise.

---

1 Advantages of MVC over Web Forms (at least 3)

✅ Interview Answer

1. **Separation of Concerns** – MVC separates Model, View, and Controller clearly.
2. **Better Testability** – MVC supports unit testing easily.
3. **No ViewState** – Reduces page size and improves performance.
4. **Full Control over HTML** – Better for SEO and frontend customization.
5. **Better Routing Control** – Clean URLs.

🧠 Easy

MVC = Clean + Testable + Lightweight

---

2 Different Return Types of Controller Action Method

✅ Interview Answer

Common return types:

- ViewResult
- PartialViewResult
- JsonResult
- ContentResult
- RedirectResult
- RedirectToRouteResult
- FileResult
- ActionResult (base class)

Example:

```
public ActionResult Index()
{
    return View();
}
```

---

3 What are Filters in MVC?

✅ Interview Answer

Filters allow executing code before or after action methods.

Types of Filters:

1. Authorization Filter
2. Action Filter
3. Result Filter
4. Exception Filter

🎯 Real Use

Logging, Authentication, Exception handling.

---

4 Authentication vs Authorization

✅ Interview Answer

- Authentication → Verifies who the user is.
- Authorization → Verifies what the user can access.

Example:
Login = Authentication
Access Admin page = Authorization

---

5 Types of Authentication in ASP.NET MVC

1. Forms Authentication
2. Windows Authentication
3. OAuth Authentication

4. JWT Authentication
5. Identity-based Authentication

---

6 What is Output Caching?

✅ Interview Answer

Output caching stores the output of a controller action to improve performance.

Example:

```
[OutputCache(Duration = 60)]
public ActionResult Index()
```

It caches result for 60 seconds.

---

7 What is Routing in MVC?

✅ Interview Answer

Routing maps incoming URL requests to controller actions.

Example:

```
/Home/Index
```

Maps to:
Controller → Home
Action → Index

---

8 What is Attribute-Based Routing?

✅ Interview Answer

Attribute routing defines routes directly on action methods using attributes.

Example:

```
[Route("students/list")]
public ActionResult GetStudents()
```

More flexible and readable.

---

9 Difference between ViewData, ViewBag & TempData

ViewData	ViewBag	TempData
Dictionary	Dynamic wrapper	Uses Session
Requires type casting	No type casting	Used across redirect
Works only current request	Same	Survives one redirect

🧠 Easy

ViewData/ViewBag → Same request
TempData → Next request

---

10 How to pass data from Controller to View?

✅ Methods:

1. ViewData
2. ViewBag
3. TempData
4. Strongly Typed Model (Best practice)

Example:

```
return View(studentModel);
```

Strongly typed models are recommended.

1️⃣1️⃣ What is Partial View?

✅ Interview Answer

Partial View is a reusable view component used to render a portion of a page.

Used for:

- Header
- Footer
- Reusable components

Example:

```
@Html.Partial("_StudentDetails")
```

🧠 Easy

Partial View = Reusable UI block

🔥 Ashok — You are now strong in:

✔ MVC
✔ SQL
✔ C# Advanced
✔ LINQ
✔ GC & Threading

Next interviewer may move to:

- Dependency Injection
- SOLID principles
- ASP.NET Core Middleware
- REST API
- JWT
- Entity Framework Core
- Microservices

Tell me next 🚀

What are Areas in MVC? Q. How Validation works in MVC? What is Data Annotation? Q. Explain the concept of MVC Scaffolding? Q. What is Bundling and Minification in MVC? Q. How to implement Security in web applications in MVC? Q. What are the events in Page Life Cycle? In which event are the controls fully loaded? Q. What is the difference between Server.Transfer() and Response.Redirect()? Q. What are the different types of Caching?

Excellent Ashok 🔥
These are mixed **MVC + Web Forms + Security + Performance** questions — very common in interviews.

Let's go step by step, interview-ready and crisp.

1️⃣ What are Areas in MVC?

✅ Interview Answer

Areas in MVC are used to organize large applications into smaller functional sections. Each area has its own:

- Controllers
- Views
- Models

It helps in modular development.

Example:

- Admin Area
- HR Area
- Sales Area

🧠 Easy

Area = Project inside a project

---

2 How Validation works in MVC? What is Data Annotation?

✅ Interview Answer

Validation in MVC works using:

- Data Annotations
- ModelState
- Client-side validation (jQuery)

Data Annotations are attributes applied to model properties for validation.

Example:

```
public class Student
{
    [Required]
    [StringLength(50)]
    public string Name { get; set; }
}
```

MVC automatically validates before action executes.

🧠 Easy

Data Annotation = Validation rules on model

---

3 What is MVC Scaffolding?

✅ Interview Answer

Scaffolding automatically generates:

- Controller
- Views
- CRUD operations

Based on model and database structure.

Used to quickly build admin panels.

🧠 Easy

Scaffolding = Auto code generator

---

4 What is Bundling and Minification?

✅ Interview Answer

Bundling combines multiple CSS/JS files into one file.

Minification removes unnecessary spaces and characters.

Benefits:

- Reduces HTTP requests
- Improves performance

🧠 Easy

Bundle = Combine
Minify = Compress

---

5 How to implement Security in MVC?

✅ Interview Answer

Security can be implemented using:

- Authentication (Forms, JWT, Identity)

- Authorization ([Authorize] attribute)
- HTTPS
- Anti-forgery tokens
- Input validation
- Role-based access
- Secure cookies

Example:

```
[Authorize(Roles = "Admin")]
```

---

6 Page Life Cycle Events (Web Forms)

✅ Interview Answer

Main events:

1. Page_PreInit
2. Page_Init
3. Page_Load
4. Control Events
5. Page_PreRender
6. Page_Unload

◈ Controls fully loaded?

👉 In **Page_Load** event.

---

7 Server.Transfer() vs Response.Redirect()

Server.Transfer()	Response.Redirect()
Server-side	Client-side
URL does not change	URL changes
Faster	Slower
Same server only	Can redirect anywhere

🧠 Easy

Transfer → Internal
Redirect → External

---

8 Types of Caching

✅ Interview Answer

1. Output Caching
2. Data Caching
3. Distributed Caching
4. In-Memory Caching
5. Browser Caching

🎯 Real Example

- Cache frequently used lookup data
- Cache dashboard results

🧠 Easy

Caching = Store result to avoid recomputation

🔥 Ashok — Now you're strong in:

✔ MVC
✔ Web Forms
✔ Performance
✔ Security
✔ SQL
✔ Core C#

Next level interview topics:

- Dependency Injection
- SOLID principles
- ASP.NET Core Middleware
- REST API design
- JWT Authentication
- Entity Framework Core
- Microservices
- Design Patterns

Tell me next 🚀

What are the types of state management? Q. Where the ViewState is stored after the page postback? Q. What are the different ways to store session state in asp.net? Q. What is cookie less session? Q. How to force all the validation controls to run in a page in web forms? Q. What are the main components of ADO.NET? Q. What is Connected architecture and Disconnected architecture? Q. What are the different Execute Methods of ADO.NET? Q. What are the Authentication techniques used to connect to SQL Server? Q. What is ORM? What are the different types of ORM? Q. What is Entity Framework?

Excellent Ashok 🔥
Now we are covering **State Management + ADO.NET + ORM + EF** — very common for .NET backend interviews.

Let's go question by question.

1️⃣ What are the types of State Management?

✅ Interview Answer

State management is used to preserve data between requests.

Two Types:

🔷 Client-Side State Management

- ViewState
- Hidden Fields
- Cookies
- Query String

◈ Server-Side State Management

- Session
- Application
- Cache

🧠 Easy

Client side → Stored in browser
Server side → Stored in server

---

2 Where is ViewState stored after postback?

✅ Interview Answer

ViewState is stored in a hidden field (`__VIEWSTATE`) on the page and sent to the client (browser). It is encoded and sent back to the server on postback.

🧠 Easy

ViewState → Hidden field in HTML

---

3 Different ways to store Session State in ASP.NET

✅ Interview Answer

Session state modes:

1. InProc (Default – stored in memory)
2. StateServer
3. SQL Server

4. Custom mode

🧠 Easy

InProc → Fast
StateServer → Separate service
SQL → Database storage

---

4 What is Cookieless Session?

✅ Interview Answer

Cookieless session stores session ID in the URL instead of cookies.

Example:

```
http://example.com/(S(sessionID))/Home
```

Used when browser does not support cookies.

---

5 How to force all validation controls to run?

✅ Interview Answer

Use:

```
Page.Validate();
```

And then check:

```
if (Page.IsValid)
```

---

6 Main Components of ADO.NET

✅ Interview Answer

1. Connection
2. Command
3. DataReader
4. DataAdapter
5. DataSet

🧠 Easy

Connection → Connect
Command → Execute
DataReader → Read
DataSet → Store

---

7 Connected vs Disconnected Architecture

✅ Connected Architecture

- Uses DataReader
- Connection remains open
- Faster
- Forward-only

✅ Disconnected Architecture

- Uses DataSet & DataAdapter
- Connection closed after fetching data
- Data manipulated offline

🧠 Easy

Connected → Live connection
Disconnected → Offline copy

---

8 Execute Methods in ADO.NET

✅ Interview Answer

1. ExecuteReader() → Returns DataReader
2. ExecuteScalar() → Returns single value
3. ExecuteNonQuery() → INSERT/UPDATE/DELETE

---

9 Authentication techniques for SQL Server

✅ Interview Answer

1. Windows Authentication
2. SQL Server Authentication

🧠 Easy

Windows login
SQL login

---

10 What is ORM?

✅ Interview Answer

ORM (Object Relational Mapping) maps database tables to C# objects.

Instead of writing SQL, we use objects.

Example:
Table → Class
Row → Object

Types of ORM

1. Full ORM (Entity Framework, NHibernate)
2. Micro ORM (Dapper)

1️⃣1️⃣ What is Entity Framework?

✅ Interview Answer

Entity Framework is a Microsoft ORM framework that allows developers to work with database using C# objects instead of SQL queries.

Features:

- LINQ support
- Change tracking
- Migrations
- Lazy loading

🧠 Easy

EF = Database using C# objects

🔥 Ashok — You are now covering:

✅ Web Forms
✅ MVC
✅ ADO.NET
✅ SQL
✅ EF
✅ State Management

Next interviewer may go into:

- EF Core tracking vs No tracking
- Code First vs DB First
- Lazy vs Eager loading
- Dependency Injection
- SOLID principles
- Design Patterns
- Middleware
- REST API
- JWT
- Microservices

Tell me next topic 🚀

How will you differentiate ADO.NET from Entity Framework? Q. How Entity Framework works? OR How to setup EF? Q. What is meant by DBContext and DBSet? Q. What are the different types of application development approaches used with EF? Q. What is the difference between LINQ to SQL and Entity Framework? Q. What is Web API? What is the purpose of Web API? Q. What are Web API advantages over WCF and web serivces? Q. What are HTTP verbs or HTTP methods? Q. What is the difference Rest API and Web API? Q. What are REST guidelines? What is the difference between Rest and Restful? Q. Is it possible to use WCF as Restful services? Q. How to consume Web API from a .NET MVC application?

Excellent Ashok 🔥
Now we're entering **EF + Web API + REST architecture** — very important for senior backend interviews.

I'll keep answers interview-ready and structured.

1️⃣ Differentiate ADO.NET and Entity Framework

✅ Interview Answer

ADO.NET	Entity Framework
Low-level data access	ORM framework
Manual SQL queries	Uses LINQ
More control	Faster development
More code	Less code
Better performance (sometimes)	Easier maintenance

🧠 Easy

ADO.NET → Manual SQL
EF → Work with objects

---

2 How does Entity Framework work?

✅ Interview Answer

Entity Framework works using ORM concept:

1. DB tables are mapped to C# classes
2. DbContext manages connection & change tracking
3. LINQ queries are converted to SQL
4. EF executes SQL and maps result back to objects

How to setup EF?

1. Install EF package (NuGet)
2. Create DbContext class
3. Create model classes
4. Configure connection string
5. Run migrations

---

3 What is DbContext and DbSet?

✅ Interview Answer

🔷 DbContext

Main class responsible for:

- Database connection
- Change tracking
- Saving changes

◈ DbSet

Represents a table in the database.

Example:

```
public class AppDbContext : DbContext
{
    public DbSet<Student> Students { get; set; }
}
```

🧠 Easy

DbContext → Database manager
DbSet → Table

---

4 Types of EF Development Approaches

1. Code First
2. Database First
3. Model First

🧠 Easy

Code First → Start from C#
DB First → Start from database

---

5 LINQ to SQL vs Entity Framework

LINQ to SQL	Entity Framework
Only SQL Server	Multiple databases
Simple mapping	Advanced mapping
Limited features	Full ORM support
Old technology	More powerful

---

6 What is Web API?

✅ Interview Answer

Web API is a framework used to build RESTful HTTP services that can be consumed by web, mobile, or desktop applications.

It returns data (JSON/XML), not views.

🎯 Purpose

To expose data/services over HTTP.

---

7 Web API advantages over WCF/Web Services

Web API	WCF
Lightweight	Complex
Supports REST easily	SOAP heavy
Works with HTTP verbs	Protocol dependent
Better for web/mobile	Enterprise level

---

8 What are HTTP verbs?

- GET → Retrieve
- POST → Create
- PUT → Update
- DELETE → Remove
- PATCH → Partial update

---

9 REST API vs Web API

✅ Interview Answer

Web API is framework.
REST is architectural style.

Web API can be RESTful.

🔟 REST Guidelines

- Use proper HTTP verbs
- Use nouns in URLs
- Stateless communication
- Return proper status codes
- Use JSON

REST vs RESTful

REST → Architecture
RESTful → API following REST rules

1️⃣1️⃣ Can WCF be RESTful?

✅ Yes

By configuring WCF with WebHttpBinding.

But Web API is preferred for REST services.

1️⃣2️⃣ How to consume Web API from MVC?

✅ Interview Answer

Ways:

1. HttpClient
2. AJAX
3. Fetch API
4. RestSharp

Example using HttpClient:

```
HttpClient client = new HttpClient();
var response = await client.GetAsync("api/students");
```

🧠 Easy

MVC → Call API using HttpClient

🔥 Ashok — You are now covering:

✔ ADO.NET
✔ EF
✔ Web API
✔ REST
✔ MVC

Next level topics interviewer may go:

- Dependency Injection
- SOLID Principles
- Middleware in ASP.NET Core
- JWT Authentication
- Microservices
- Design Patterns
- Exception Handling Best Practices
- Performance tuning in ASP.NET Core

Tell me next 🚀

How to consume Web API from a .NET MVC application? Q. What is the difference between Web API and MVC Controller? Q. What are the types of authentication techniques in web api? Q. What is Basic Authentication in Web API? Q. What is API Key Authentication in Web API? Q. What is Token based authentication? Q. What is JWT Authentication? Q. What are the parts of JWT token? Q. Where JWT token reside in the request? Q. How to test Web API? What are

the tools? Q. What are main Return Types supported in Web API? Q. What is the difference between HTTPResponseMessage and IHttpActionResult? Q. What is the difference between IActionResult and IHttpActionResult? Q. What is Content Negotiation in Web API? Q. What is MediaTypeFormatter class in Web API? Q. What are Response Codes in Web API?

Excellent Ashok 🔥
Now we are in **Web API + Authentication + HTTP pipeline deep interview zone** — very important for backend roles.

Let's go one by one, interview-ready and structured.

1️⃣ How to consume Web API from .NET MVC application?

✅ Interview Answer

We can consume Web API in MVC using:

1. **HttpClient (Recommended)**
2. AJAX (jQuery / Fetch)
3. RestSharp

Example using HttpClient:

```
HttpClient client = new HttpClient();
client.BaseAddress = new Uri("https://localhost:5001/");
var response = await client.GetAsync("api/students");

if (response.IsSuccessStatusCode)
{
    var data = await response.Content.ReadAsAsync<List<Student>>();
}
```

🧠 Easy

MVC → Call API using HttpClient

---

2 Difference between Web API and MVC Controller

MVC Controller	Web API Controller
Returns Views (HTML)	Returns Data (JSON/XML)
Used for UI apps	Used for services
ActionResult	IHttpActionResult / IActionResult
Mainly browser-based	Web, mobile, external apps

---

3 Types of Authentication in Web API

1. Basic Authentication
2. API Key Authentication
3. Token-based Authentication
4. JWT Authentication
5. OAuth

---

4 What is Basic Authentication?

✅ Interview Answer

Basic Authentication sends username and password encoded in Base64 format in HTTP header.

Header Example:

```
Authorization: Basic base64(username:password)
```

Not secure unless used with HTTPS.

---

5 What is API Key Authentication?

✅ Interview Answer

API Key Authentication uses a unique key sent in request header or query string to authorize client.

Example:

```
x-api-key: 123456
```

Used in public APIs.

---

6 What is Token-Based Authentication?

✅ Interview Answer

User logs in → Server generates token → Client sends token in every request → Server validates token.

Stateless authentication.

---

7 What is JWT Authentication?

✅ Interview Answer

JWT (JSON Web Token) is a secure token-based authentication mechanism where user identity and claims are stored inside a signed token.

Used in modern APIs.

---

8 Parts of JWT Token

JWT has 3 parts:

1. Header
2. Payload
3. Signature

Structure:

```
Header.Payload.Signature
```

🧠 Easy

JWT = Header + Data + Signature

---

9 Where does JWT token reside in request?

✅ Interview Answer

Usually stored in:

```
Authorization: Bearer <token>
```

Sent in HTTP header.

---

10 How to test Web API?

✅ Tools

1. Postman
2. Swagger
3. Curl
4. Fiddler
5. Thunder Client

---

11 Main Return Types in Web API

- IActionResult
- IHttpActionResult
- HttpResponseMessage
- Specific types (Ok(), NotFound(), etc.)

---

12 Difference: HttpResponseMessage vs IHttpActionResult

HttpResponseMessage	IHttpActionResult
Full control	Cleaner & simpler
Manual response building	Framework handles response
Older approach	Recommended approach

---

13 IActionResult vs IHttpActionResult

IActionResult	IHttpActionResult
Used in ASP.NET Core	Used in ASP.NET Web API
Modern	Older version
More flexible	Limited to Web API

---

14 What is Content Negotiation?

✅ Interview Answer

Content negotiation determines response format (JSON/XML) based on:

- Accept header in request

Example:

```
Accept: application/json
```

Server returns JSON.

---

15 What is MediaTypeFormatter?

✅ Interview Answer

MediaTypeFormatter converts .NET objects into specific formats like JSON or XML.

Example:

- JsonMediaTypeFormatter
- XmlMediaTypeFormatter

1⃣6⃣ Response Codes in Web API

Common HTTP status codes:

- 200 OK
- 201 Created
- 204 No Content
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 500 Internal Server Error

🧠 Easy

2xx → Success
4xx → Client error
5xx → Server error

🔥 Ashok — You have now covered:

✔ MVC
✔ Web API
✔ Authentication
✔ JWT
✔ EF
✔ SQL
✔ C#
✔ GC & Threading

Next interviewer level topics:

- Dependency Injection
- SOLID principles
- Middleware in ASP.NET Core
- Microservices architecture
- Design patterns (Singleton, Factory, Repository)
- Clean Architecture
- Exception handling best practices
- Performance tuning

Tell me next 🚀

What is .NET Core? Q. What is .NET Standard? Q. What are the advantages of .NET Core over .NET framework? Q. What is the role of Program.cs file in ASP.NET Core? Q. What is the role of ConfigureServices method? Q. What is the role of Configure method? Q. Describe the complete Request Processing Pipeline for ASP.NET Core MVC? Q. What is the difference between .NET Core and .NET 5? Q. What is Metapackage? What is the name of Metapackage provided by ASP.NET Core? Q. What is Dependency Injection? Q. How to implement Dependency injection in .NET Core?

Excellent Ashok 🔥
Now we are in **ASP.NET Core + Modern .NET architecture** — very important for senior backend interviews.

Let’s go step by step, interview-ready and clear.

1️⃣ What is .NET Core?

✅ Interview Answer

.NET Core is a cross-platform, open-source, high-performance framework developed by Microsoft for building modern applications such as web APIs, cloud apps, microservices, and console applications.

It runs on Windows, Linux, and macOS.

🧠 Easy

.NET Core = Cross-platform modern .NET

---

2 What is .NET Standard?

✅ Interview Answer

.NET Standard is a specification that defines a set of APIs that all .NET implementations must support.

It ensures code sharing across:

- .NET Framework
- .NET Core
- Xamarin

🧠 Easy

.NET Standard = Common API contract

---

3 Advantages of .NET Core over .NET Framework

✅ Interview Answer

1. Cross-platform support
2. Better performance
3. Lightweight & modular
4. Built-in Dependency Injection
5. Better for microservices & cloud

🧠 Easy

Core = Faster + Cross-platform + Modern

---

4 Role of Program.cs in ASP.NET Core

✅ Interview Answer

Program.cs is the entry point of ASP.NET Core application.

It:

- Builds the host
- Configures services
- Configures middleware pipeline
- Starts the application

In .NET 6+, minimal hosting model handles startup inside Program.cs.

---

5 Role of ConfigureServices()

✅ Interview Answer

ConfigureServices() is used to register services into the Dependency Injection container.

Example:

```
services.AddControllers();
services.AddDbContext<AppDbContext>();
```

🧠 Easy

ConfigureServices → Register services

---

6 Role of Configure()

✅ Interview Answer

Configure() method defines the HTTP request pipeline using middleware.

Example:

```
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

🧠 Easy

Configure → Setup middleware

---

7 Complete Request Processing Pipeline (ASP.NET Core MVC)

✅ Interview Answer

1. Request comes to Kestrel server
2. Middleware pipeline executes in order
3. Routing middleware identifies endpoint
4. Model binding happens
5. Controller action executes
6. Result executed (View/JSON)
7. Response sent back

🧠 Easy

Request → Middleware → Routing → Controller → Response

---

8 Difference between .NET Core and .NET 5

✅ Interview Answer

.NET 5 is the unified platform that replaces .NET Core and .NET Framework.

.NET Core → Initial cross-platform version
.NET 5 → Unified modern .NET

After .NET 5, naming changed to .NET 6, 7, 8.

---

9 What is Metapackage?

✅ Interview Answer

Metapackage is a package that includes multiple related packages.

In ASP.NET Core:

```
Microsoft.AspNetCore.App
```

It includes essential ASP.NET Core libraries.

---

10 What is Dependency Injection?

✅ Interview Answer

Dependency Injection (DI) is a design pattern where dependencies are injected into a class instead of being created inside the class.

It promotes loose coupling and testability.

🧠 Easy

Don’t create dependency → Inject it

---

11 How to implement DI in .NET Core?

✅ Steps

1. Create Interface
2. Create Implementation
3. Register service in ConfigureServices()
4. Inject via constructor

Example:

```
services.AddScoped<IStudentService, StudentService>();
```

Controller:

```
public class StudentController : Controller
{
    private readonly IStudentService _service;

    public StudentController(IStudentService service)
    {
        _service = service;
    }
}
```

🧠 Easy

Register → Inject → Use

🔥 Ashok — Now you're strong in:

✔ ASP.NET Core
✔ DI
✔ Middleware
✔ Web API
✔ EF
✔ SQL
✔ C# Advanced

Next interviewer likely moves to:

- Service lifetimes (Scoped/Transient/Singleton)
- Middleware deep dive
- Logging in .NET Core

- Configuration in .NET Core
- SOLID principles
- Clean Architecture
- Microservices
- Docker & Kubernetes
- Caching in .NET Core
- Performance tuning

Tell me next topic 🚀

What are the advantages of Dependency Injection in .NET Core? Q. How to use Dependency Injection in Views in ASP.NET Core? Q. What are the types of Service Lifetimes of an object/ instance in ASP.NET Core? Q. What is AddSingleton, AddScoped and AddTransient method? Q. What is Middleware in ASP.NET Core? What is custom middleware? Q. How ASP.NET Core Middleware is different from HttpModule? Q. What is Request Delegate in .NET Core? Q. What is Run(), Use() and Map() method? Q. What are the types of Hosting in ASP.NET Core? What is In process and Out of process hosting? Q. What is Kestrel? What is the difference between Kestrel and IIS? Q. What is Routing? Explain attribute routing in ASP.NET Core? Q. Explain default project structure in ASP.NET Core application?

Excellent Ashok 🔥
Now we are in **ASP.NET Core Deep Architecture + Middleware + Hosting** — this is senior-level interview territory.

Let's go step by step, crisp and interview-ready.

---

1 Advantages of Dependency Injection in .NET Core

✅ Interview Answer

1. Loose coupling between classes
2. Improves unit testing
3. Better maintainability
4. Centralized service management
5. Supports lifetime management

🧠 Easy

DI = Flexible + Testable + Maintainable

---

2 How to use Dependency Injection in Views?

✅ Interview Answer

In Razor view, use `@inject` directive.

Example:

```
@inject IStudentService studentService

<h1>@studentService.GetCount()</h1>
```

Used rarely — mainly for helper services.

---

3 Service Lifetimes in ASP.NET Core

✅ Three Types:

1. Singleton
2. Scoped
3. Transient

---

4 AddSingleton, AddScoped, AddTransient

Method	Lifetime
AddSingleton	One instance for entire application
AddScoped	One instance per request
AddTransient	New instance every time requested

🧠 Easy

Singleton → App lifetime
Scoped → Request lifetime
Transient → Every time

---

5 What is Middleware in ASP.NET Core?

✅ Interview Answer

Middleware are components that handle HTTP requests and responses in the request pipeline.

Each middleware can:

- Process request
- Call next middleware
- Stop pipeline

Custom Middleware Example:

```
public class MyMiddleware
{
    private readonly RequestDelegate _next;

    public MyMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task Invoke(HttpContext context)
    {
        await context.Response.WriteAsync("Hello");
        await _next(context);
    }
}
```

---

6 Middleware vs HttpModule

Middleware	HttpModule
Used in ASP.NET Core	Used in .NET Framework
Simple pipeline	Complex event-based model
Ordered execution	Event-based execution

---

7 What is RequestDelegate?

✅ Interview Answer

RequestDelegate represents a function that processes HTTP request.

Signature:

```
Task RequestDelegate(HttpContext context)
```

It passes control to next middleware.

---

8 Run(), Use(), Map()

✅ Use()

Calls next middleware.

```
app.Use(async (context, next) =>
{
    await next();
});
```

✅ Run()

Terminal middleware (does not call next).

```
app.Run(async context =>
{
    await context.Response.WriteAsync("Done");
});
```

✅ Map()

Branches pipeline based on path.

```
app.Map("/admin", adminApp =>
{
});
```

---

9 Types of Hosting in ASP.NET Core

✅ Two Types:

1. In-Process Hosting
2. Out-of-Process Hosting

In-Process Hosting

- App runs inside IIS worker process
- Better performance

Out-of-Process Hosting

- IIS acts as reverse proxy
- App runs inside Kestrel

---

10 What is Kestrel?

✅ Interview Answer

Kestrel is a cross-platform web server used by ASP.NET Core.

It handles HTTP requests directly.

Kestrel vs IIS

Kestrel	IIS
Cross-platform	Windows only
Lightweight	Full web server
Used with reverse proxy	Enterprise server

---

11 What is Routing?

✅ Interview Answer

Routing maps incoming HTTP request to controller actions.

Example:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Attribute Routing in ASP.NET Core

```
[Route("api/students")]
public class StudentController : Controller
{
    [HttpGet]
    public IActionResult Get() { }
}
```

---

12 Default Project Structure (ASP.NET Core)

✅ Main Components

- Program.cs → Entry point
- appsettings.json → Configuration
- Controllers → Controllers
- Models → Business entities
- Views → Razor views
- wwwroot → Static files
- Middleware → Custom middleware
- Services → Business logic

🧠 Easy

Program → Start
Controllers → Handle
Views → UI
Models → Data
wwwroot → Static files

🔥 Ashok — You have now covered:

✔ ASP.NET Core Architecture
✔ Middleware
✔ DI
✔ Hosting
✔ Routing
✔ Kestrel

You are now in **Senior Backend Developer Level** 🚀

Next advanced topics interviewer may go:

- SOLID principles
- Clean Architecture
- Repository pattern
- Unit of Work
- Microservices
- Caching in ASP.NET Core
- Logging & Serilog
- Performance tuning
- Deadlocks
- Thread safety
- Memory management deep dive

Tell me next topic 🔥

How ASP.NET Core serve static files? Q. What are the roles of Appsettings.Json and Launchsetting.Json file in ASP.NET Core? Q. What are the various techniques to save configuration settings in ASP.NET Core? Q. What is CORS? Why is CORS restriction is required? How to fix CORS error? Q. What is In-Memory caching & Distributed Caching? Q. How to handle errors in ASP.NET Core? Q. What are Razor pages in .NET Core? Q. What are

SOLID Principles? What is the difference between SOLID Principles and Design Patterns? Q. What is Single Responsibility Principle? Q. What is Open-closed Principle? Q. What is Liskov Substitution Principle? Q. What is Interface Segregation Principle? Q. What is Dependency Inversion Principle? Q. What is DRY principle?

Excellent Ashok 🔥
Now we're in **ASP.NET Core + Caching + CORS + SOLID Principles** — very important for senior backend interviews.

Let's go step by step.

---

1 How ASP.NET Core serves static files?

✅ Interview Answer

ASP.NET Core serves static files from the **wwwroot** folder using Static File Middleware.

To enable:

```
app.UseStaticFiles();
```

Static files include:

- CSS
- JS
- Images
- HTML

Without `UseStaticFiles()`, static files won't load.

🧠 Easy

wwwroot + UseStaticFiles() = Static file serving

---

2 Role of appsettings.json & launchSettings.json

✅ appsettings.json

- Stores application configuration
- DB connection strings
- API keys
- Logging settings

Example:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=..."
  }
}
```

✅ launchSettings.json

- Used only in development
- Defines environment variables
- Sets application URL & ports

Not used in production.

🧠 Easy

appsettings → App configuration
launchSettings → Dev environment config

---

3 Techniques to save configuration in ASP.NET Core

1. appsettings.json
2. Environment variables
3. User secrets
4. Azure Key Vault
5. Command line arguments

---

4 What is CORS?

✅ Interview Answer

CORS (Cross-Origin Resource Sharing) is a security feature that restricts web pages from making requests to a different domain.

Browsers block cross-origin requests by default.

Why required?

To prevent:

- Cross-site attacks
- Unauthorized access

How to fix CORS error?

In Program.cs:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAll",
        policy => policy.AllowAnyOrigin()
                        .AllowAnyMethod()
                        .AllowAnyHeader());
});

app.UseCors("AllowAll");
```

🧠 Easy

Different domain → Blocked
Enable CORS → Allow access

---

5 In-Memory vs Distributed Caching

In-Memory Caching

- Stored in application memory
- Fast
- Lost when app restarts

Example:

```
services.AddMemoryCache();
```

Distributed Caching

- Stored outside app (Redis, SQL Server)
- Shared across multiple servers
- Suitable for load-balanced apps

🧠 Easy

Memory → Local
Distributed → Shared

---

6 How to handle errors in ASP.NET Core?

Techniques:

1. UseExceptionHandler()
2. Custom middleware
3. Try-Catch blocks
4. Logging (Serilog)
5. DeveloperExceptionPage (dev only)

Example:

```
app.UseExceptionHandler("/Home/Error");
```

---

7 What are Razor Pages?

✅ Interview Answer

Razor Pages is a page-based programming model in ASP.NET Core.

Each page has:

- .cshtml file
- PageModel class (.cshtml.cs)

Better for small apps and simple CRUD.

🧠 Easy

MVC → Controller-based
Razor → Page-based

---

8 What are SOLID Principles?

SOLID is a set of 5 object-oriented design principles.

SOLID vs Design Patterns

SOLID → Guidelines for writing good code
Design Patterns → Reusable solutions to common problems

---

9 Single Responsibility Principle (SRP)

A class should have only one reason to change.

Example:
Separate:

- Business logic
- Logging

- Data access

---

10 Open-Closed Principle (OCP)

Software should be open for extension but closed for modification.

Use:

- Interfaces
- Polymorphism

---

11 Liskov Substitution Principle (LSP)

Derived class should be replaceable with base class without breaking functionality.

---

12 Interface Segregation Principle (ISP)

Clients should not depend on methods they don't use.

Instead of one big interface → multiple small interfaces.

---

13 Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Use interfaces + DI.

---

14 What is DRY Principle?

DRY = Don’t Repeat Yourself

Avoid code duplication.

Reusable methods, services, helpers.

🔥 Ashok — Now you have covered:

✔ ASP.NET Core
✔ Middleware
✔ Caching
✔ CORS
✔ Razor
✔ SOLID

You are now at **Senior Backend Developer Level** 🚀

Next possible advanced topics:

- Clean Architecture
- Repository & Unit of Work
- Design Patterns (Singleton, Factory, Strategy)
- Microservices
- Docker
- Logging & Serilog
- Performance optimization
- Deadlocks & Thread safety
- EF Core tracking vs NoTracking
- Transactions in EF

Tell me next 🔥

What are Design Patterns and what problem they solve? Q. What are the types of Design Patterns? Q. What are Creational Design Patterns? Q. What are Structural Design Patterns? Q. What are Behavioral Design Patterns? Q. What is Singleton Design Pattern? Q. How to make singleton pattern thread safe? Q. What is Factory pattern? Why to use factory pattern? Q. How to implement Factory method pattern? Q. What is Abstract Factory pattern?

Excellent Ashok 🔥
Now we are in **Design Patterns – Senior Level Interview Zone**.
I’ll explain clearly and interview-ready.

---

1 What are Design Patterns? What problem do they solve?

✅ Interview Answer

Design Patterns are reusable solutions to common software design problems. They provide proven best practices to make code more maintainable, flexible, scalable, and loosely coupled.

They solve problems like:

- Object creation complexity
- Tight coupling
- Code duplication
- Scalability issues

🧠 Easy

Design Pattern = Proven solution template

---

2 Types of Design Patterns

There are 3 main categories:

1. Creational
2. Structural
3. Behavioral

---

3 What are Creational Design Patterns?

✅ Interview Answer

Creational patterns deal with object creation mechanisms.

Examples:

- Singleton
- Factory
- Abstract Factory
- Builder
- Prototype

🧠 Easy

Creational → How objects are created

---

4 What are Structural Design Patterns?

✅ Interview Answer

Structural patterns deal with object composition and structure.

Examples:

- Adapter
- Decorator
- Facade
- Proxy
- Composite

🧠 Easy

Structural → How classes are organized

---

5 What are Behavioral Design Patterns?

✅ Interview Answer

Behavioral patterns focus on communication between objects.

Examples:

- Strategy
- Observer
- Command
- Mediator
- Chain of Responsibility

🧠 Easy

Behavioral → How objects communicate

---

6 What is Singleton Design Pattern?

✅ Interview Answer

Singleton ensures that only one instance of a class exists throughout the application and provides a global access point.

Used for:

- Logging
- Configuration
- Caching
- DB connection manager

Basic Singleton:

```
public class Singleton
{
    private static Singleton _instance;

    private Singleton() { }

    public static Singleton GetInstance()
    {
        if (_instance == null)
            _instance = new Singleton();

        return _instance;
```

```
    }
}
```

---

7 How to make Singleton Thread-Safe?

✅ Best Way (Recommended)

Using `Lazy<T>`:

```
public sealed class Singleton
{
    private static readonly Lazy<Singleton> _instance =
        new Lazy<Singleton>(() => new Singleton());

    public static Singleton Instance => _instance.Value;

    private Singleton() { }
}
```

🧠 Easy

Use Lazy<T> for thread safety

---

8 What is Factory Pattern? Why use it?

✅ Interview Answer

Factory Pattern creates objects without exposing the creation logic to the client.

It promotes:

- Loose coupling
- Centralized object creation
- Open-Closed Principle

🎯 Real Example

Payment system:

- CreditCardPayment
- UPIPayment
- NetBankingPayment

Factory decides which object to create.

---

9 How to implement Factory Method Pattern?

✅ Example

Step 1: Interface

```
public interface IPayment
{
    void Pay();
}
```

Step 2: Concrete Classes

```
public class CreditCardPayment : IPayment
{
    public void Pay() { }
}
```

Step 3: Factory

```
public class PaymentFactory
{
    public static IPayment GetPayment(string type)
    {
        if (type == "CreditCard")
            return new CreditCardPayment();

        throw new ArgumentException("Invalid Type");
    }
}
```

🧠 Easy

Factory = Object creator

🔟 What is Abstract Factory Pattern?

✅ Interview Answer

Abstract Factory provides an interface for creating families of related objects without specifying their concrete classes.

It is factory of factories.

🎯 Example

Car company:

Factory 1 → BMW (Engine + Tyre)
Factory 2 → Audi (Engine + Tyre)

Each factory creates related objects.

🧠 Easy

Factory → Creates one object
Abstract Factory → Creates group of related objects

🔥 Ashok — Now you are covering:

✔ SOLID
✔ DI
✔ Middleware
✔ Design Patterns
✔ EF
✔ Web API
✔ SQL
✔ C#

You are now officially in **Senior Backend Developer Level** 🚀

Next high-level interview topics:

- Repository Pattern
- Unit of Work
- Strategy Pattern
- Observer Pattern
- Clean Architecture
- Microservices
- CQRS
- MediatR
- DDD
- Performance tuning in .NET
- Thread safety deep dive